AWS Secure Web Application Deployment

with High Availability and Cloud Security

Table of Contents

Table of Contents 2
1. Project Overview 3
2. Objectives 3
3. Architecture Diagram 4
4. Architecture Explanation 5
4.1 Public Layer 5
4.2 Private Web Layer 5
4.3 Private Database Layer 5
5. AWS Services Used 6
6. VPC Configuration 7
7. Subnets 8
8. Route Tables 9
9. Security Groups 10
10. IAM Role 12
11. Bastion Host 13
12. Custom AMI and Web Server Build 14
13. Launch Template 15
14. Auto Scaling Group 16
15. Load Balancer 17
16. AWS WAF 18
17. Amazon RDS 18
18. AWS Secrets Manager 19
19. AWS KMS 20
20. Amazon CloudWatch 21
21. Amazon SNS 21
22. AWS CloudTrail 22
23. Amazon S3 23
24. Traffic Flow 23
Application Request Flow 23
Administrative Access Flow 23
25. Security Best Practices Applied 23
26. Challenges Faced 24
27. Future Improvements 24
28. Conclusion 24

1. Project Overview

This project demonstrates the deployment of a secure, three-tier web application on Amazon Web Services, designed around the AWS Well-Architected Framework's security and reliability pillars. The infrastructure separates the application into public, private-web, and private-database layers across two Availability Zones, achieving both network isolation and high availability.

The environment includes a custom VPC with public and private subnets, a Bastion Host for controlled administrative access, an internet-facing Application Load Balancer protected by AWS WAF, an Auto Scaling Group of Windows/IIS web servers built from a custom AMI, a Multi-AZ Amazon RDS for MySQL database, IAM least-privilege roles, AWS Secrets Manager for credential storage, and a full observability stack using CloudWatch, SNS, CloudTrail, and S3.

2. Objectives

- Design and deploy a secure, segmented AWS network using a custom VPC.
- Separate public, private-web, and private-database resources using subnets and route tables.
- Provide controlled administrative access through a Bastion Host instead of exposing servers directly.
- Deploy a highly available, internet-facing Application Load Balancer protected by AWS WAF.
- Achieve elasticity and fault tolerance using a Launch Template and Auto Scaling Group.
- Store the application database on Multi-AZ Amazon RDS for MySQL inside a private subnet.
- Protect application secrets using AWS Secrets Manager and IAM roles instead of hard-coded credentials.
- Encrypt data at rest using AWS KMS.
- Implement centralized logging and monitoring with CloudTrail, CloudWatch, SNS, and S3.
- Follow the principle of least privilege across all IAM roles and security groups.

3. Architecture Diagram

The diagram below illustrates the complete deployment: traffic flow from the client through AWS WAF, the Application Load Balancer, the Auto Scaling web tier, and into the Multi-AZ RDS database, alongside the supporting security, identity, and monitoring services.

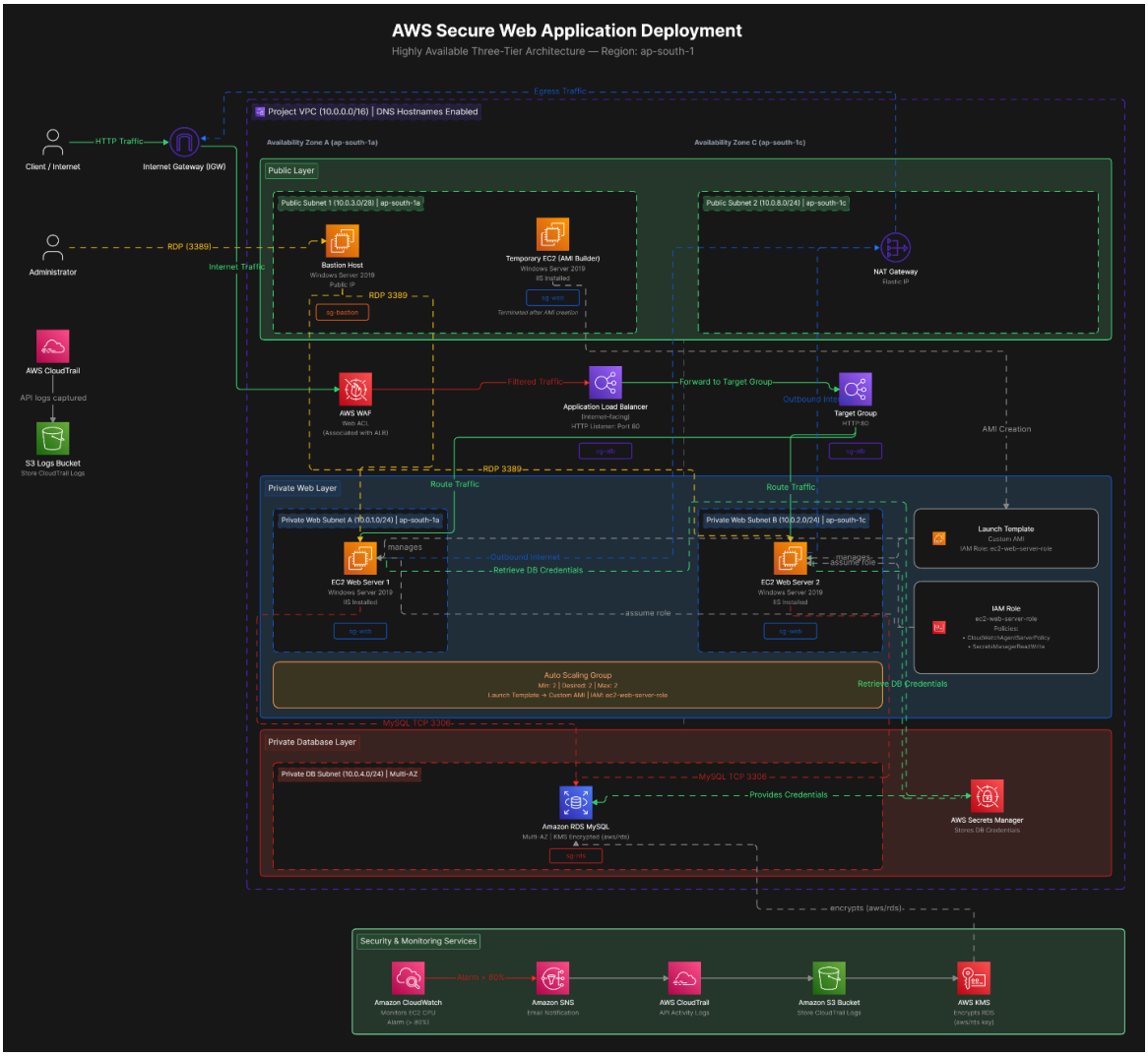


Figure 1 — AWS Secure Web Application Deployment: complete architecture

4. Architecture Explanation

4.1 Public Layer

The public layer hosts every component that needs to communicate directly with the internet.

- Internet Gateway — attaches the VPC to the internet and is the only entry/exit point for public traffic.
- Public Subnet A & B — host internet-facing resources across two Availability Zones for resilience.
- Bastion Host — a Windows Server instance with a public IP that administrators RDP into before reaching private servers, so no web server is ever exposed directly to the internet.
- Temporary EC2 (AMI Builder) — a short-lived instance used only to install IIS and prepare the application, then captured as a custom AMI and terminated.
- NAT Gateway — lets instances in the private subnets reach the internet for updates without being reachable from it.
- Application Load Balancer — the internet-facing entry point for application traffic, distributing requests across both Availability Zones.
- AWS WAF — a Web ACL attached to the ALB that filters malicious or unwanted traffic before it ever reaches the application.

4.2 Private Web Layer

The private web layer runs the application itself, fully isolated from direct internet access.

- Launch Template — defines the custom AMI, instance type, IAM role, and security group used for every new web server.
- Auto Scaling Group — keeps a minimum, desired, and maximum of two healthy IIS web servers running across both Availability Zones.
- EC2 Web Server 1 & 2 — Windows Server 2019 instances running IIS, reachable only from the ALB (HTTP) and the Bastion Host (RDP).
- IAM Role (ec2-web-server-role) — grants the web servers permission to push metrics to CloudWatch and read secrets from Secrets Manager, without embedding credentials on the instance.

4.3 Private Database Layer

The database layer is the most restricted tier in the architecture.

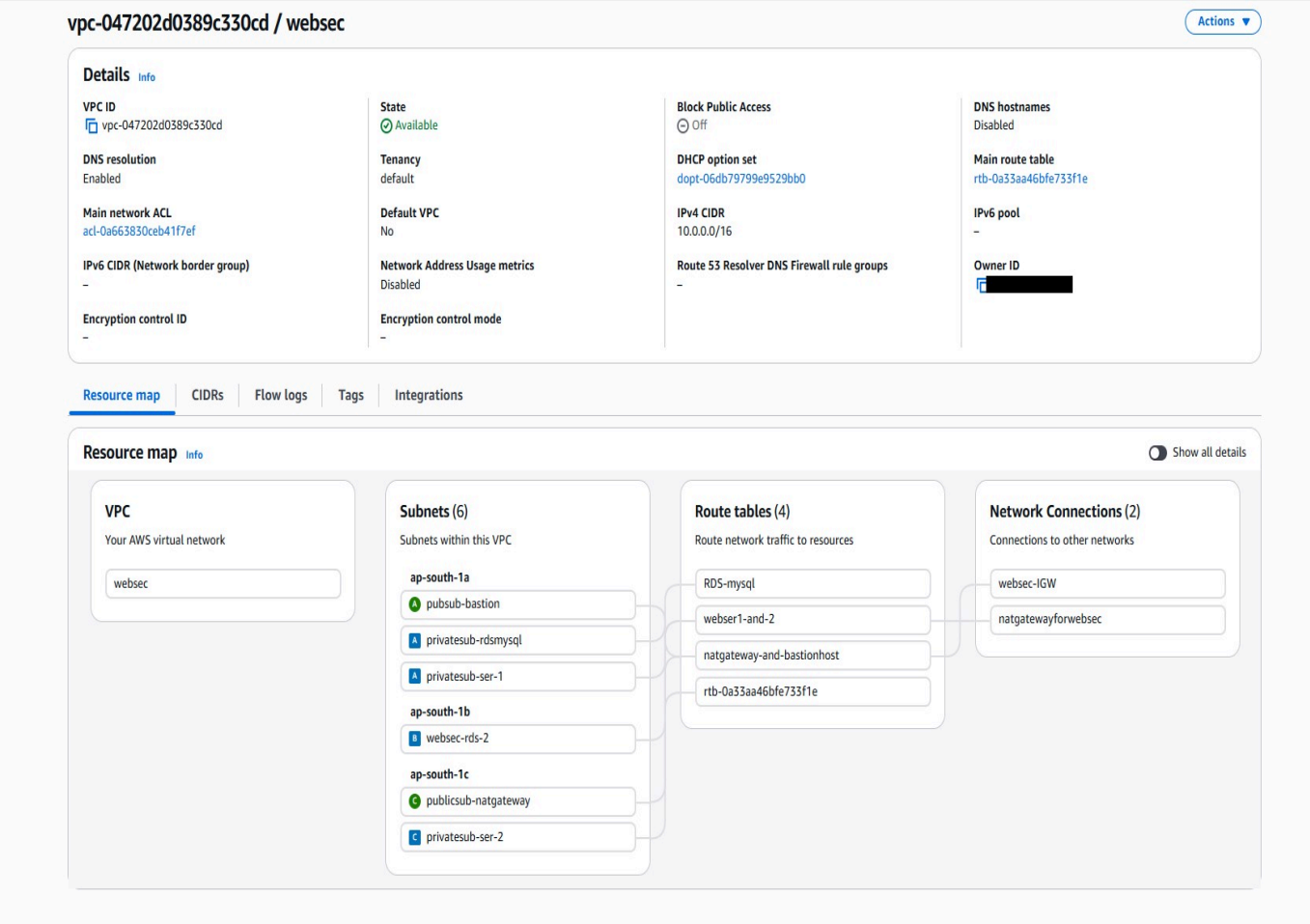
- Amazon RDS for MySQL — deployed Multi-AZ for automatic failover, placed in a private subnet with no public access.
- Private DB Subnet — accepts inbound MySQL traffic only from the web tier security group.
- AWS KMS Encryption — encrypts the RDS storage at rest using a dedicated aws/rds key.

5. AWS Services Used

Service	Purpose
Amazon VPC	Network isolation across public and private subnets
Amazon EC2	Hosts the Bastion Host and IIS web servers
Application Load Balancer (ALB)	Distributes incoming HTTP traffic across web servers
AWS WAF	Filters malicious traffic before it reaches the ALB
Auto Scaling Group (ASG)	Maintains web server availability and elasticity
IAM	Least-privilege roles and permissions for EC2 instances
AWS Secrets Manager	Securely stores and serves database credentials
Amazon RDS (MySQL)	Multi-AZ relational database for the application
AWS KMS	Encrypts RDS storage at rest
NAT Gateway	Provides outbound-only internet access for private subnets
Amazon CloudWatch	Monitors EC2 CPU utilization and raises alarms
Amazon SNS	Sends email notifications for CloudWatch alarms
AWS CloudTrail	Logs all API activity across the account
Amazon S3	Stores CloudTrail audit logs

6. VPC Configuration

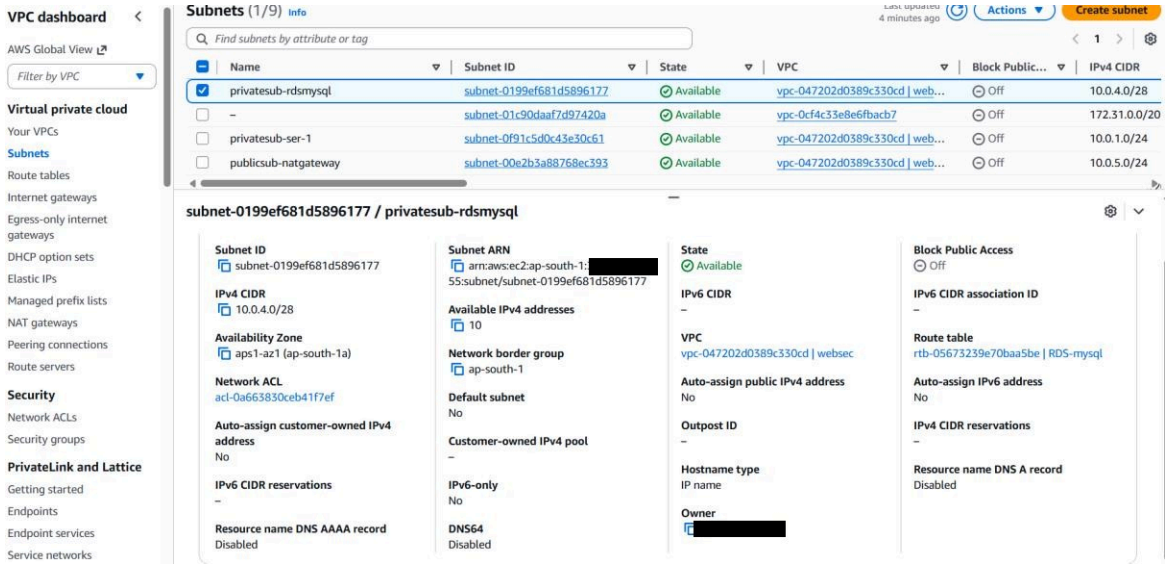
The project VPC was created with a 10.0.0.0/16 CIDR block and DNS hostnames enabled, spanning two Availability Zones (ap-south-1a and ap-south-1c) in the Asia Pacific (Mumbai) region for resilience against single-AZ failure.



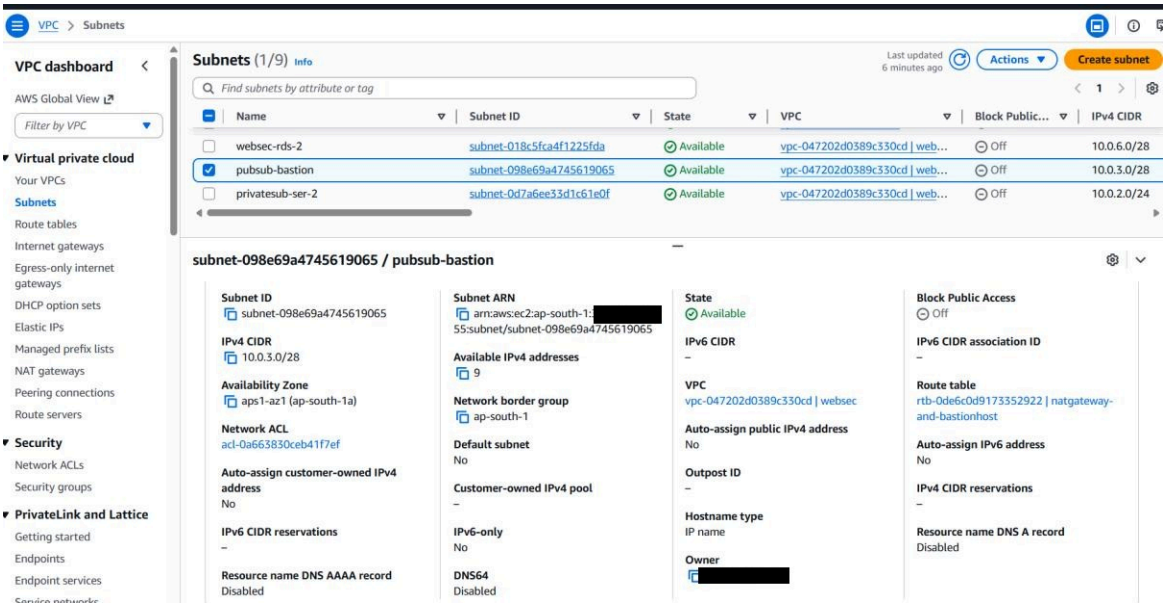
7. Subnets

Each tier of the application sits in its own subnet so that network access can be controlled independently at every layer:

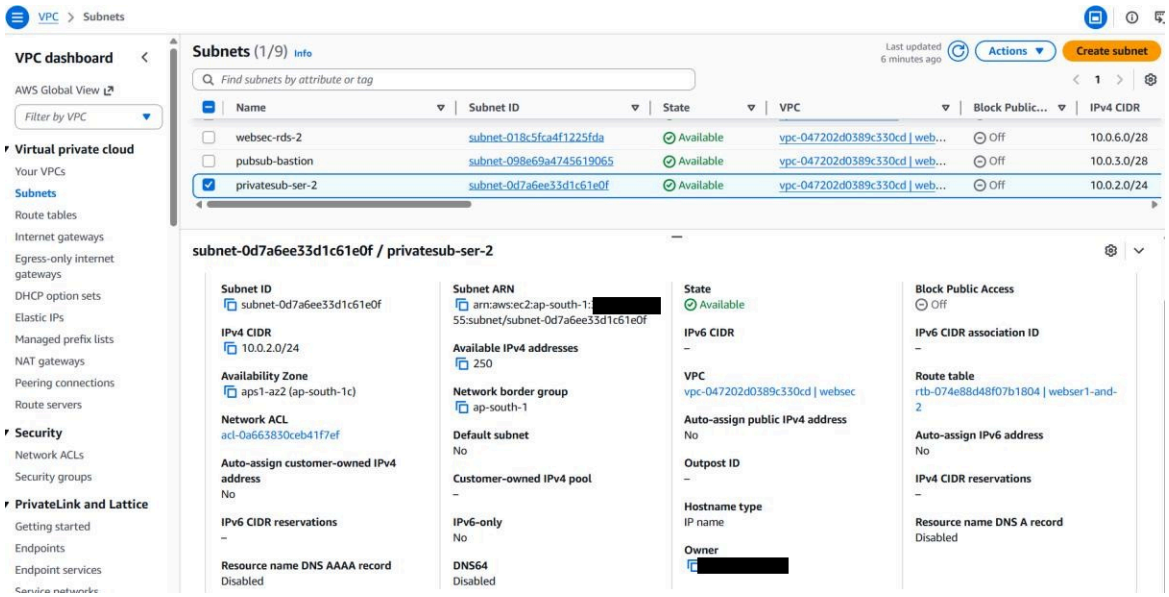
- Public Subnets — host the Bastion Host, NAT Gateway, and Application Load Balancer.
- Private Web Subnets — host the IIS web server EC2 instances, reachable only through the ALB and Bastion.
- Private DB Subnet — hosts the Multi-AZ RDS MySQL instance, reachable only from the web tier.



Subnet list — public and private subnets across the VPC



Public subnet detail — publicsub-bastion



Private subnet detail — webser-1 / privatesub-ver-1

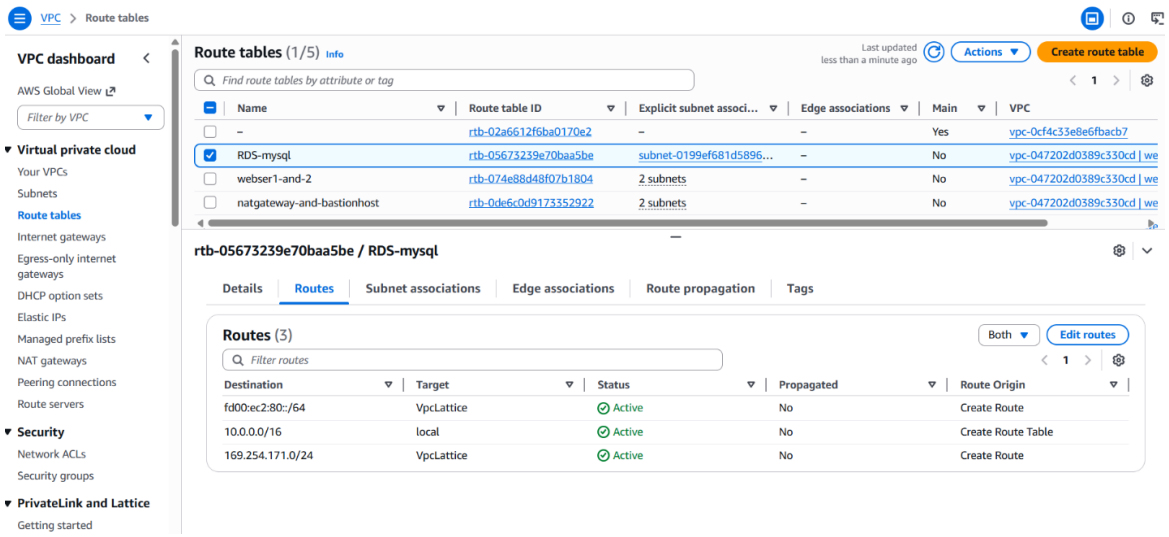
8. Route Tables

Three route tables enforce the traffic boundaries between tiers:

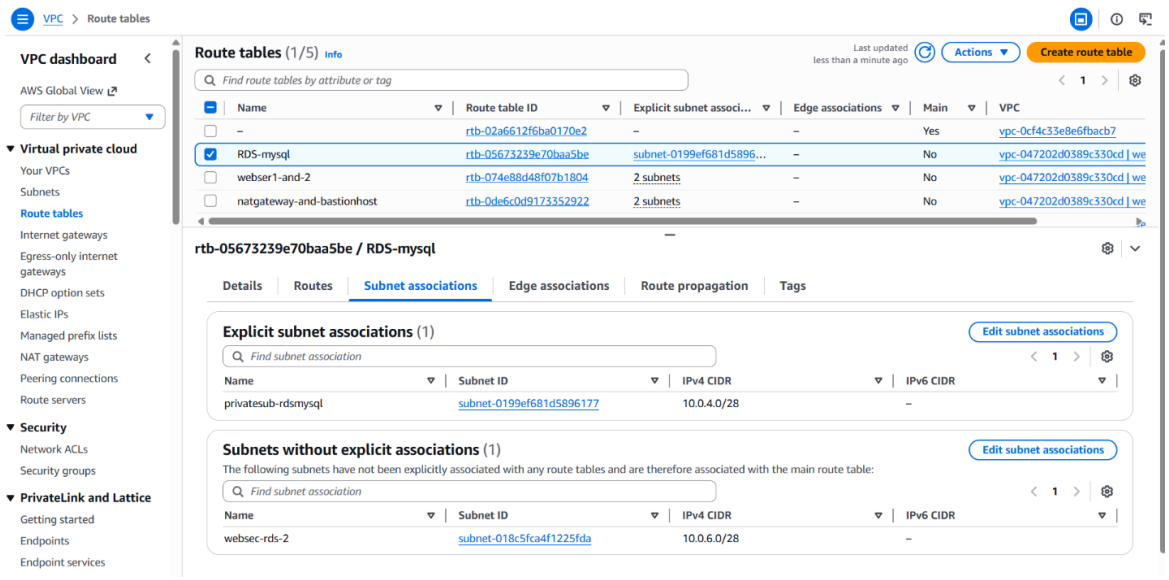
Public Route Table — 0.0.0.0/0 → Internet Gateway

Private Web Route Table — 0.0.0.0/0 → NAT Gateway

Private DB Route Table — local route only (no internet path)



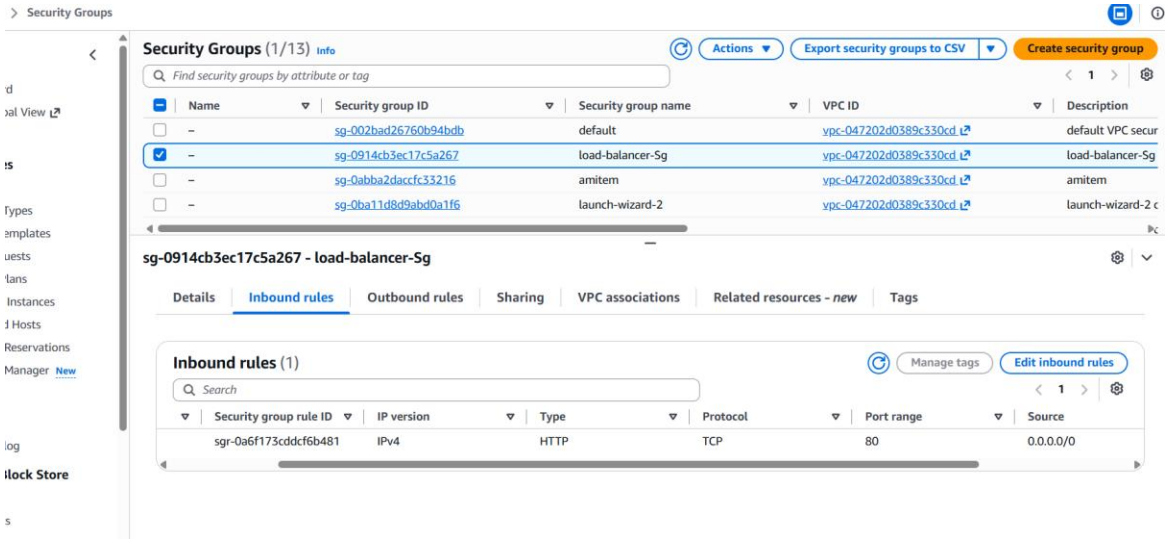
Route tables overview, including the RDS-mysql local-only table



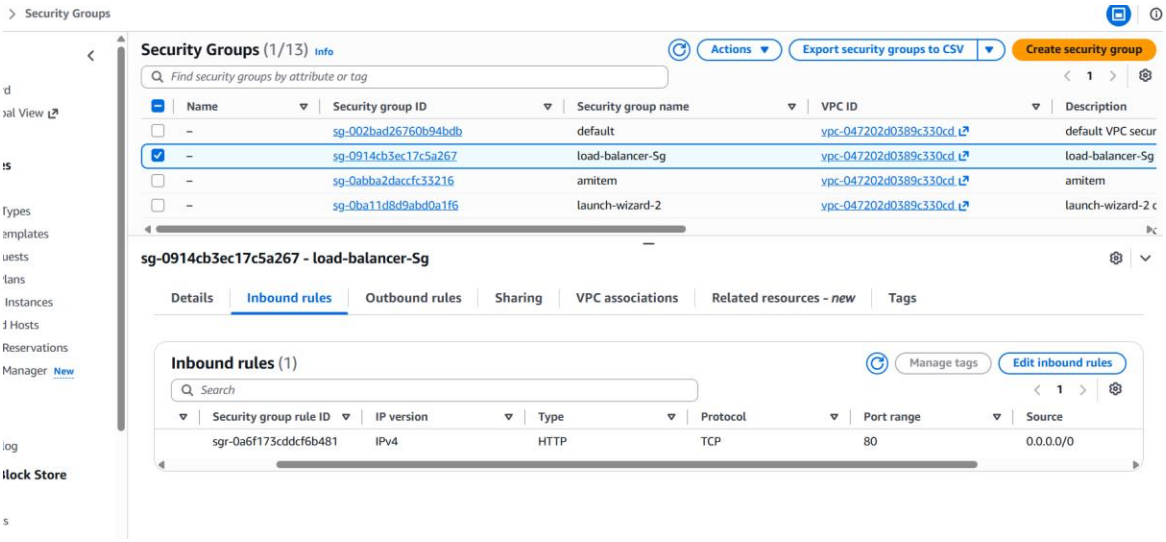
Private DB route table — subnet associations

9. Security Groups

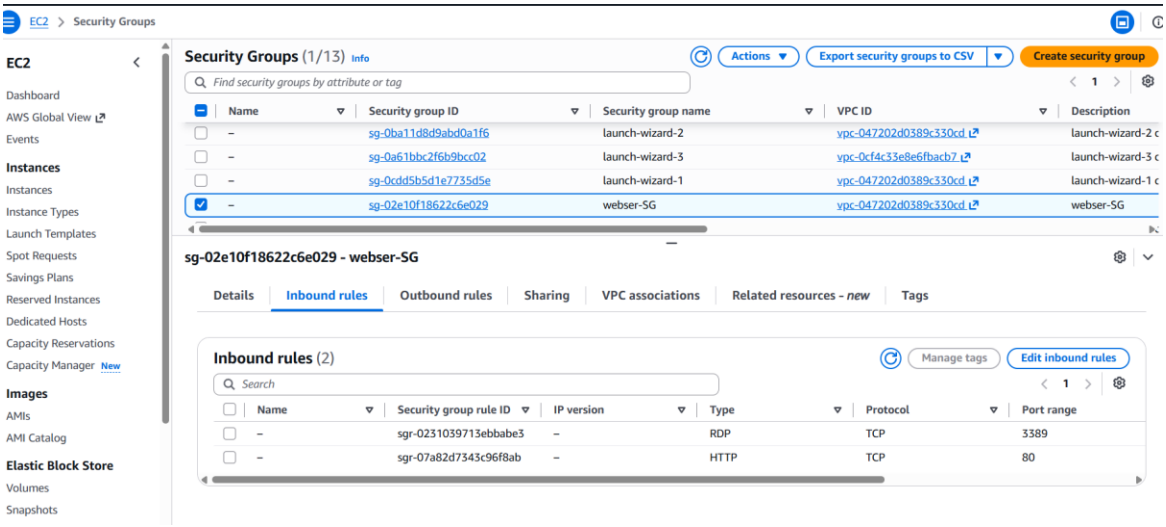
Security Group	Inbound Rules
sg-bastion	TCP 3389 (RDP) from administrator IP only
sg-alb	TCP 80 (HTTP) from 0.0.0.0/0
sg-web	TCP 80 from sg-alb; TCP 3389 (RDP) from sg-bastion
sg-rds	TCP 3306 (MySQL) from sg-web (and optionally sg-bastion)



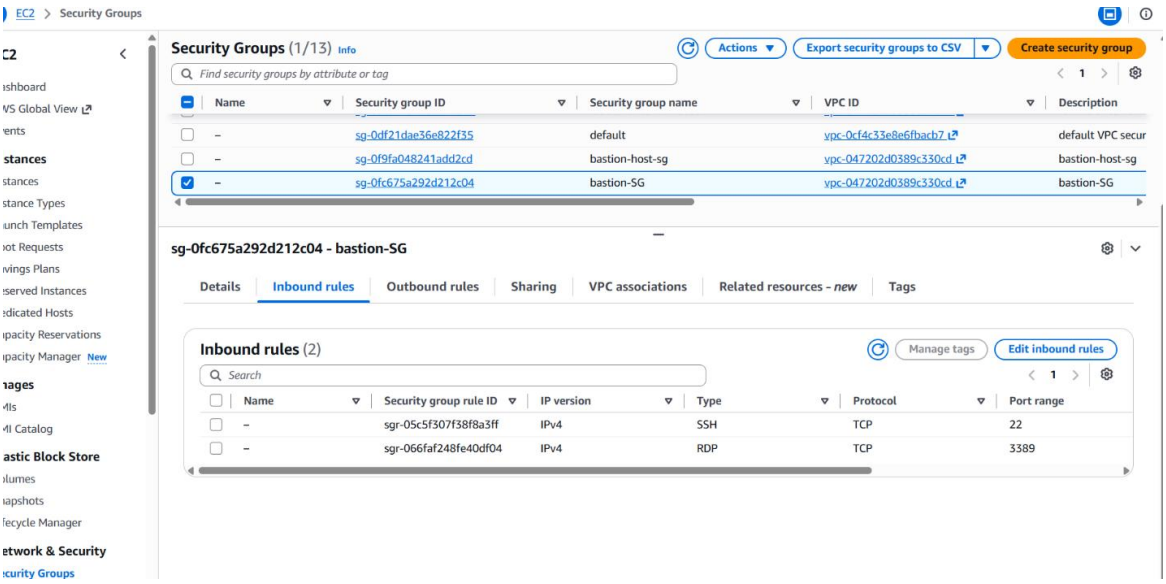
Security groups overview



load-balancer-sg — inbound rule (HTTP from internet)



web-SG — inbound rules (HTTP from ALB, RDP from Bastion)

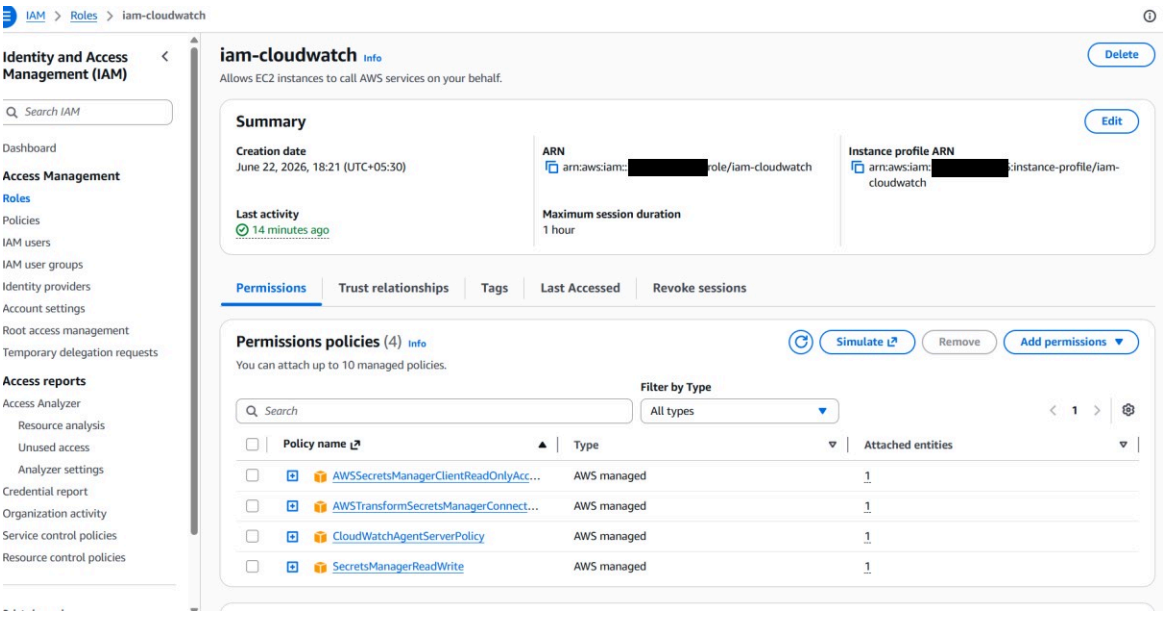


bastion-SG — inbound RDP rule

10. IAM Role

The EC2 web servers run under a dedicated IAM role rather than embedded credentials. The role is attached to the instance profile and grants exactly two managed policies:

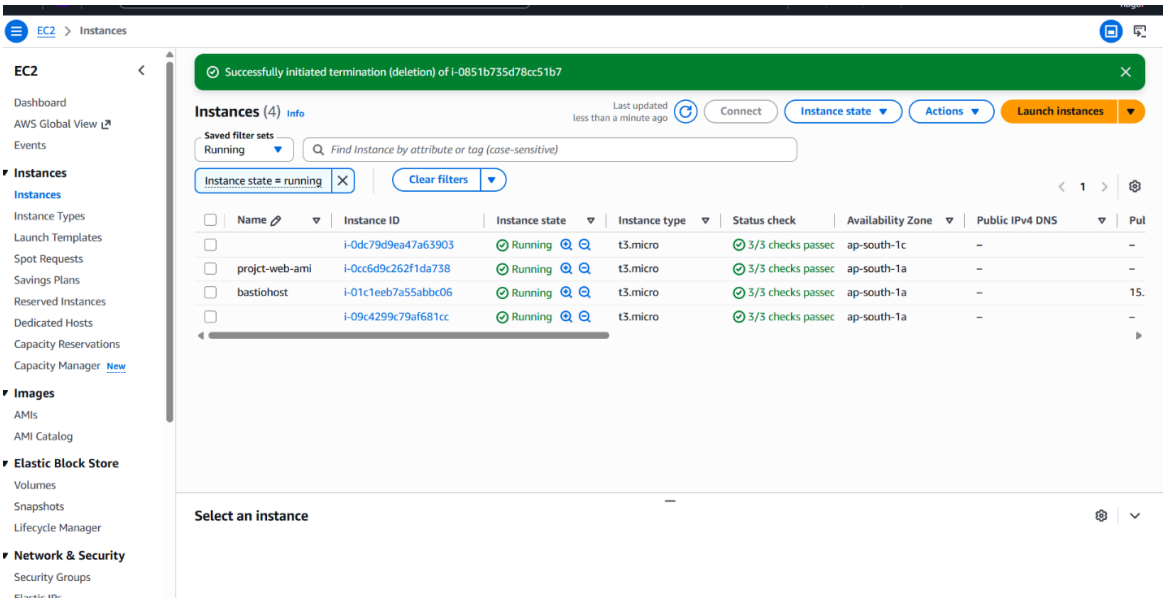
- CloudWatchAgentServerPolicy — allows the instance to publish custom metrics and logs to CloudWatch.
- SecretsManagerReadWrite — allows the instance to retrieve database credentials from Secrets Manager at runtime.



IAM role — attached permissions policies

11. Bastion Host

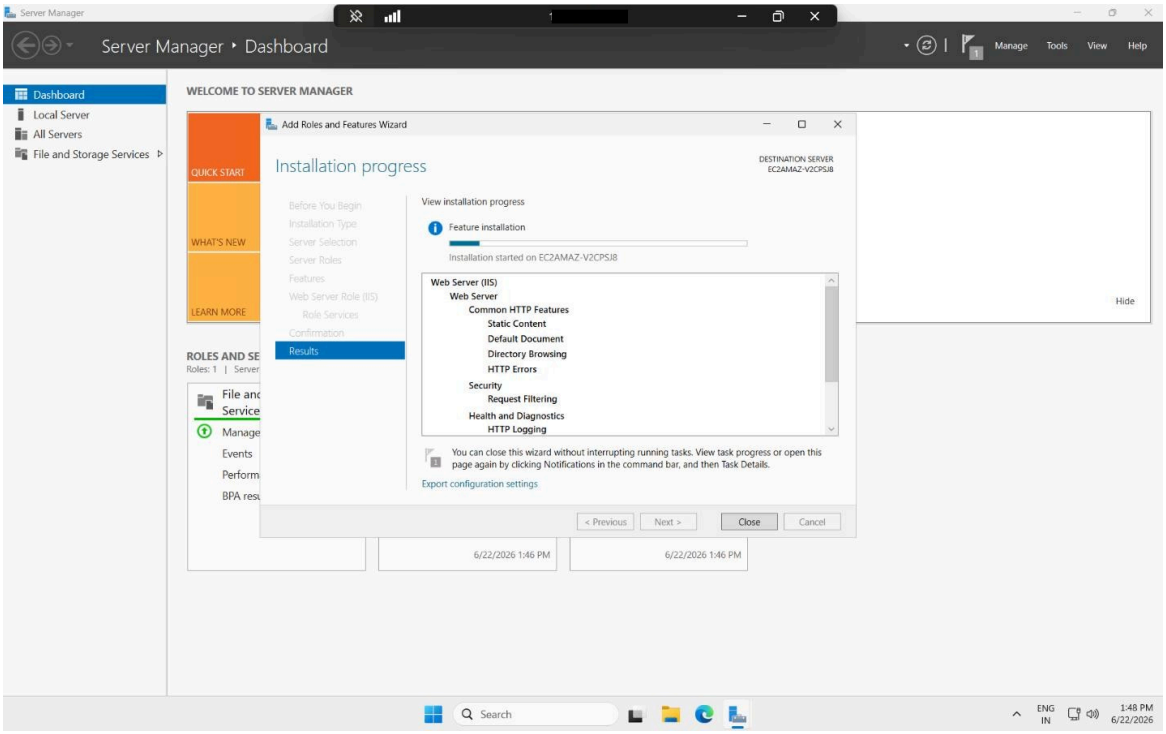
The Bastion Host is a Windows Server 2019 instance with a public IP, used exclusively for administrative RDP access. It is the only public entry point into the private network, and its security group restricts inbound RDP to a single trusted IP address.



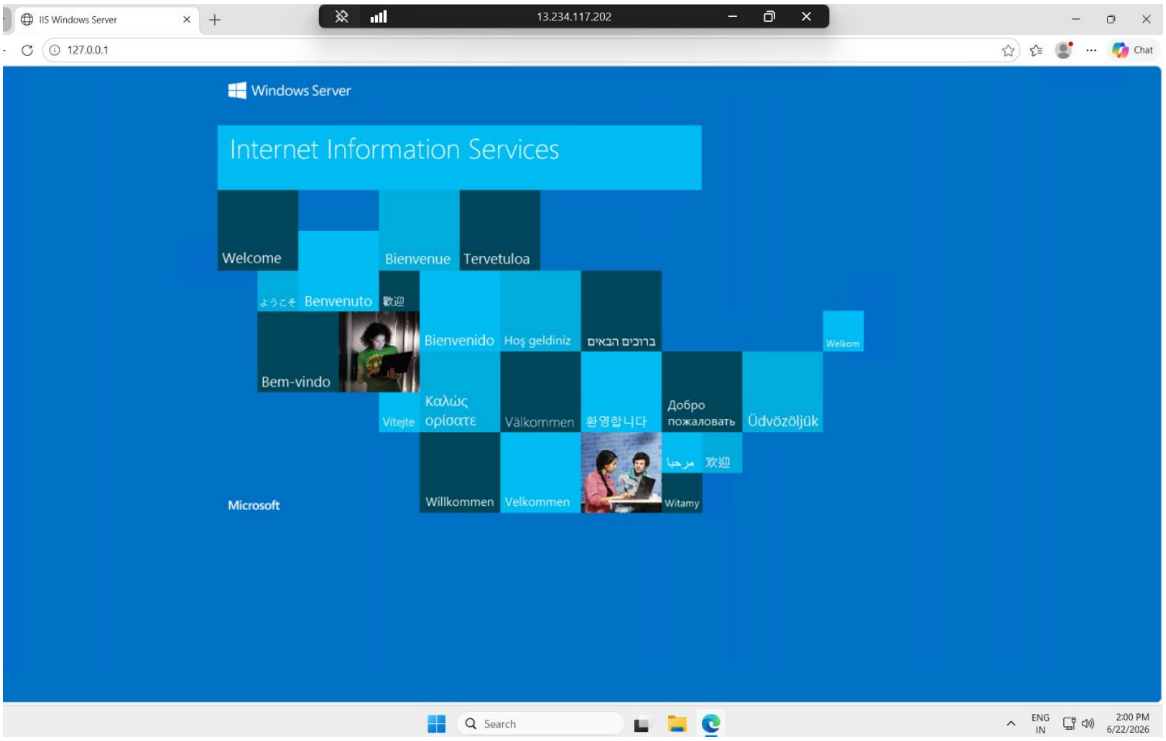
Bastion host instance — details

12. Custom AMI and Web Server Build

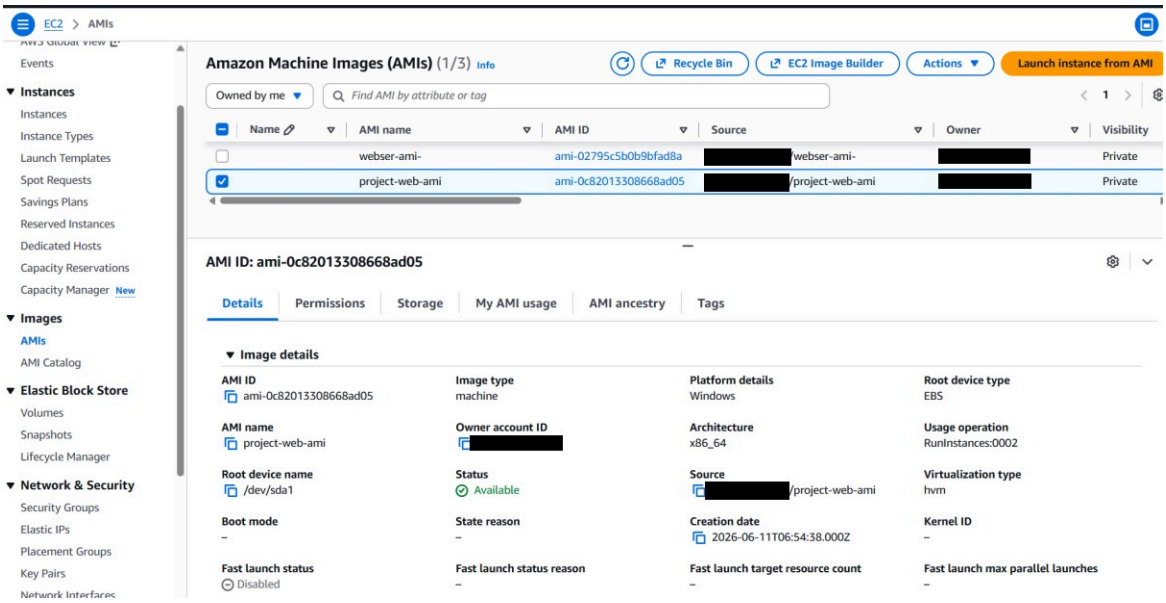
A temporary EC2 instance was used to install Windows Server 2019, enable IIS, and deploy the application before being captured as a custom Amazon Machine Image. This AMI is what the Launch Template later uses to launch identical, pre-configured web servers.



Server Manager — installing the IIS web server role



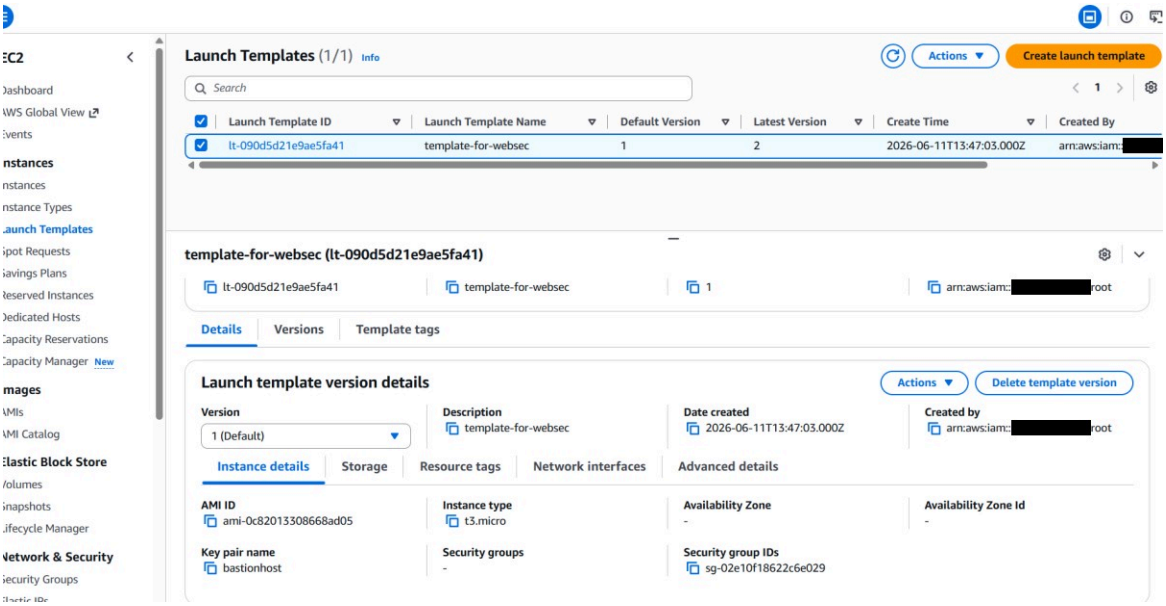
Internet Information Services (IIS) running on the instance



Custom AMI created from the AMI Builder instance

13. Launch Template

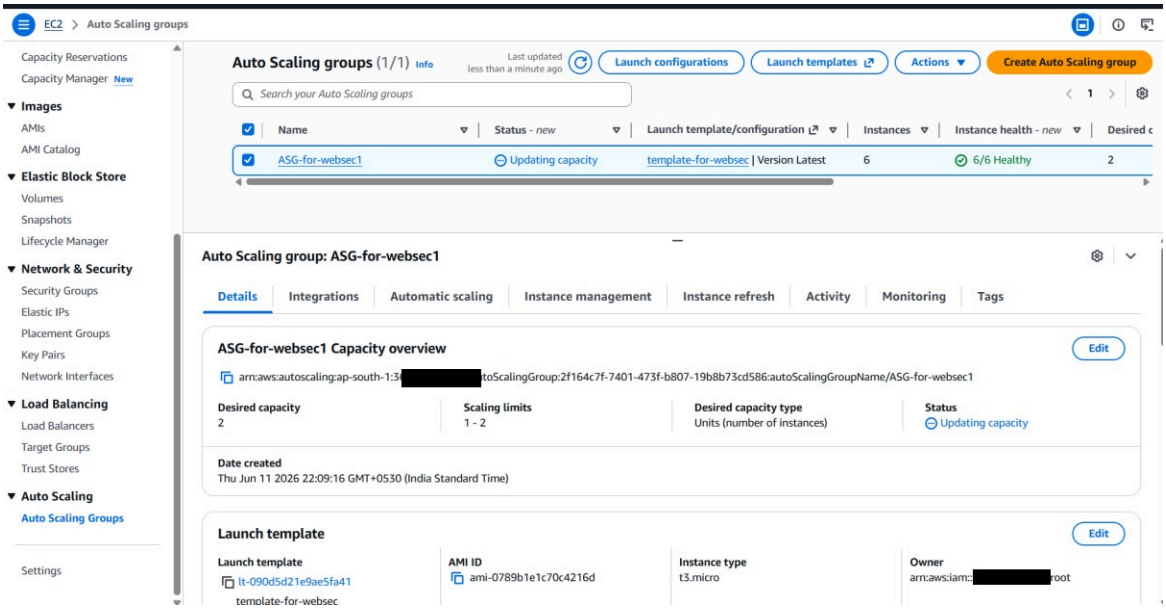
The Launch Template defines every parameter a new web server instance needs: the custom AMI, instance type, IAM instance profile, and security group, so that Auto Scaling can launch consistent, ready-to-serve instances on demand.



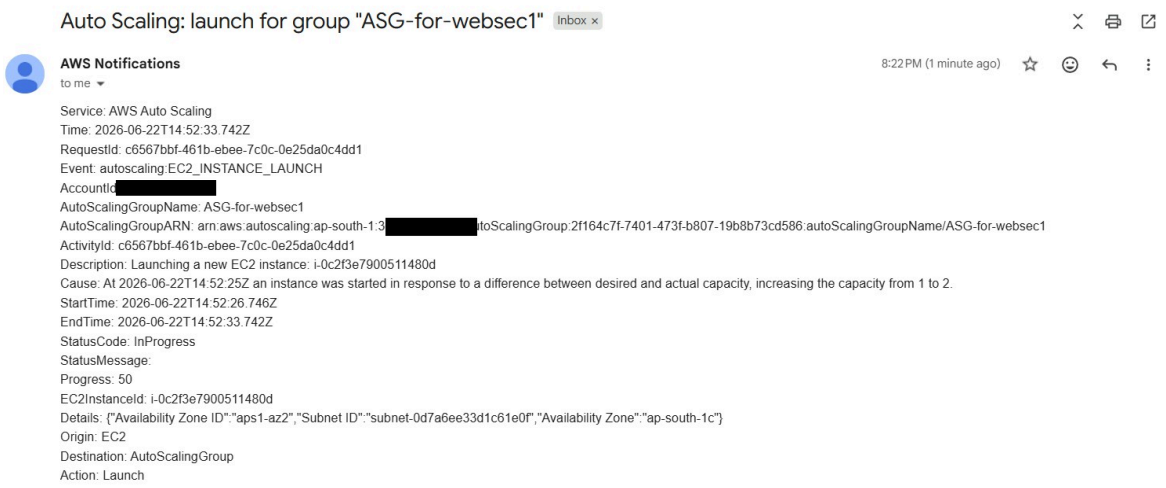
Launch Template — template details

14. Auto Scaling Group

The Auto Scaling Group is configured with a minimum, desired, and maximum capacity of two instances, spread across both Availability Zones, so the application keeps running even if an entire zone becomes unavailable.



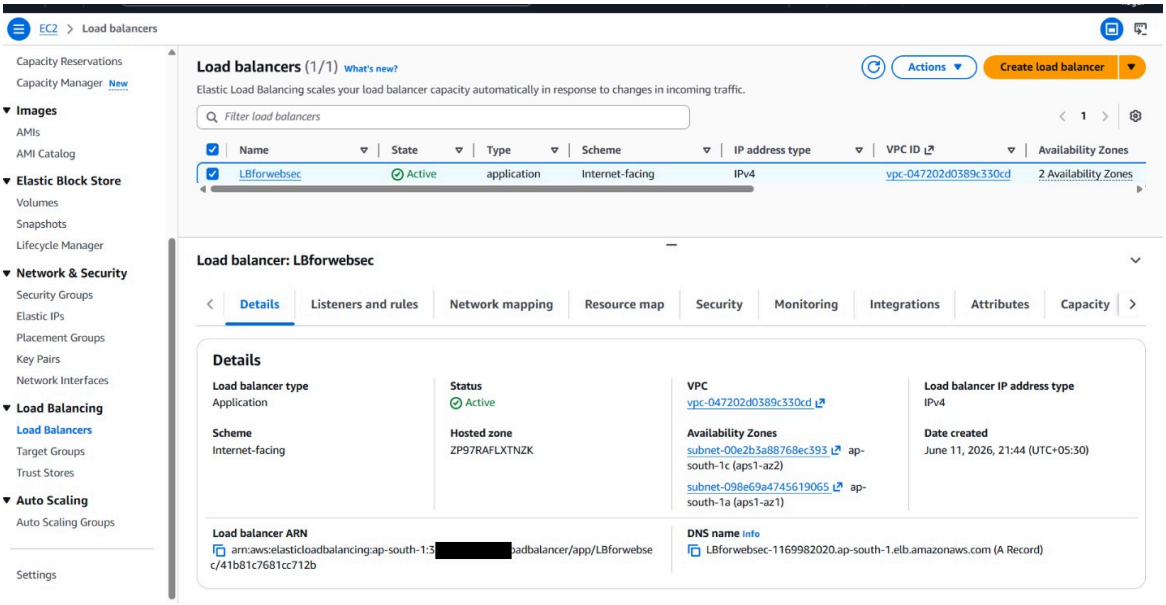
Auto Scaling group — capacity overview



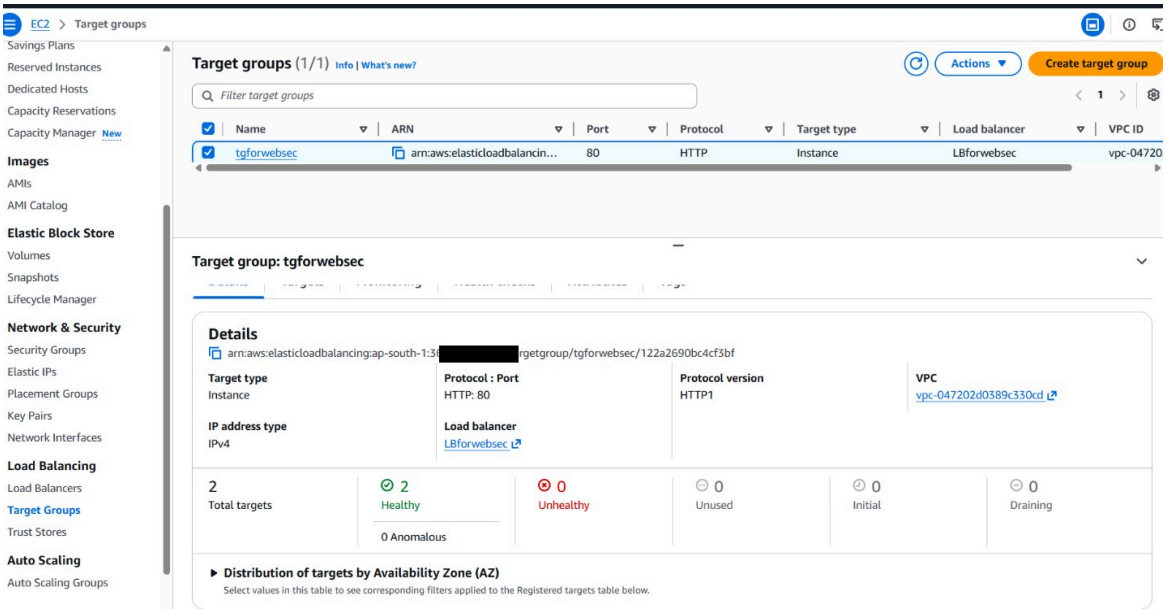
Auto Scaling launch notification for ASG-for-websec1

15. Load Balancer

An internet-facing Application Load Balancer listens on HTTP port 80 and forwards traffic to a target group containing both web server instances, performing health checks to route around any unhealthy instance.



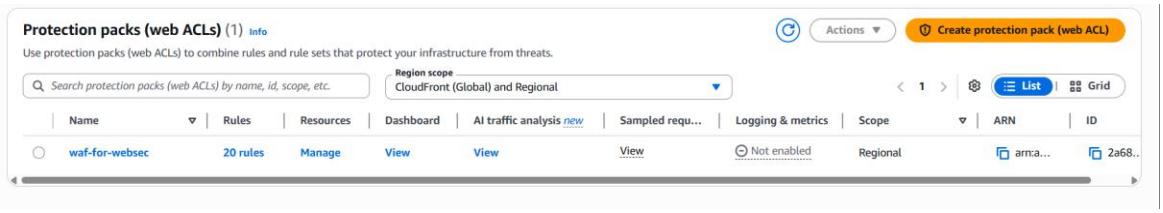
Application Load Balancer — details



Target group — registered web server targets

16. AWS WAF

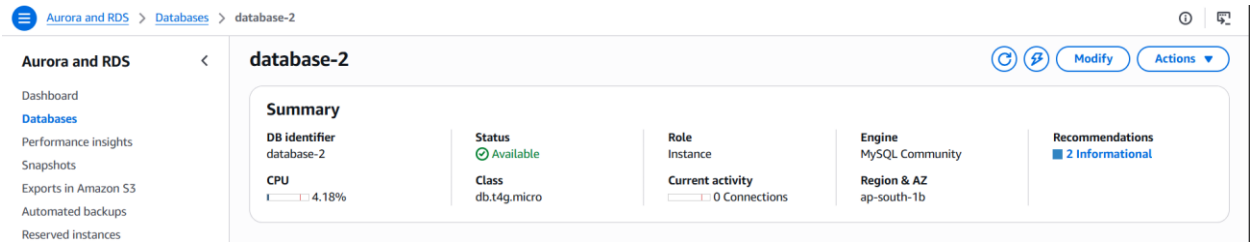
AWS WAF is associated directly with the ALB and inspects every request before it reaches the application, blocking common web exploits and unwanted traffic patterns using managed rule sets.



AWS WAF — Web ACL protection packs

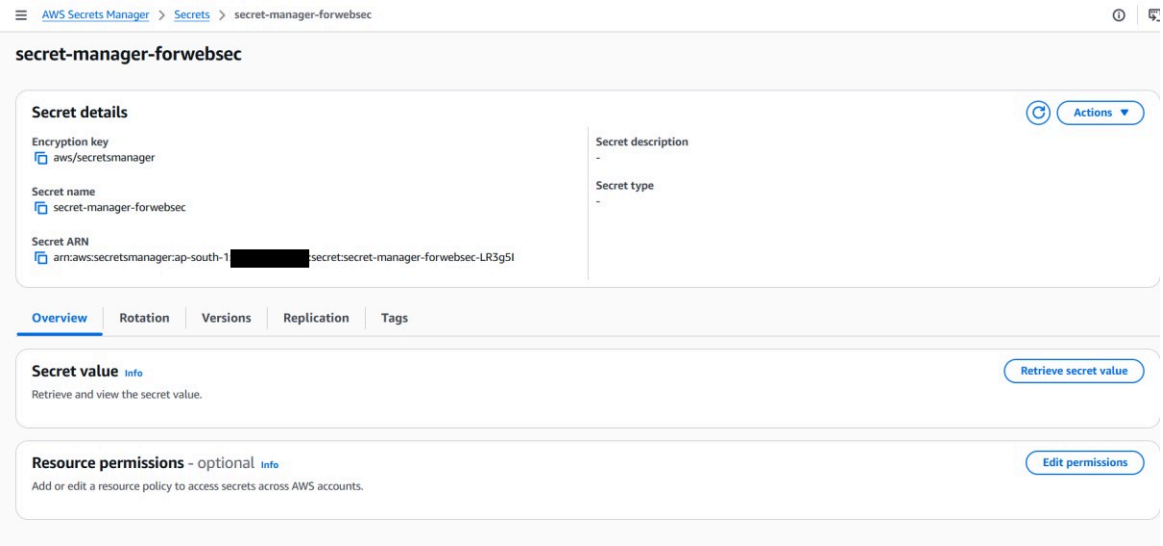
17. Amazon RDS

Amazon RDS for MySQL hosts the application's database within the private database subnet. The database is accessible only from the web server security group over **TCP port 3306** and has **no public endpoint**, preventing direct internet access. Storage encryption at rest is enabled using the **AWS-managed KMS key (aws/rds)**, protecting the database, automated backups, and snapshots without requiring manual key management.



18. AWS Secrets Manager

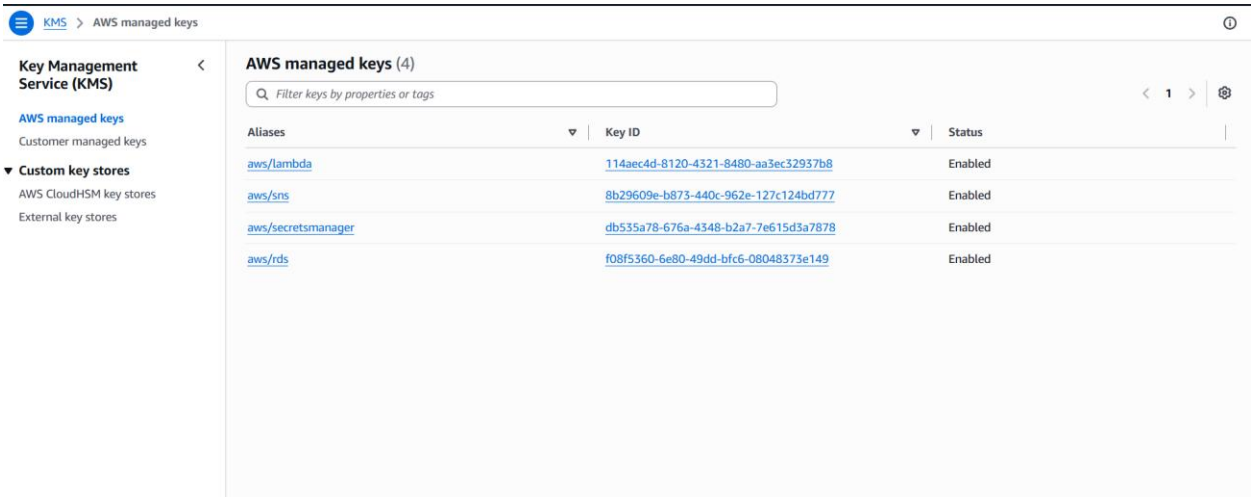
Database credentials are never stored on the web servers. Instead, the application retrieves the username and password at runtime from AWS Secrets Manager, using the IAM role attached to each EC2 instance.



Secrets Manager — secret-manager-forwebsec

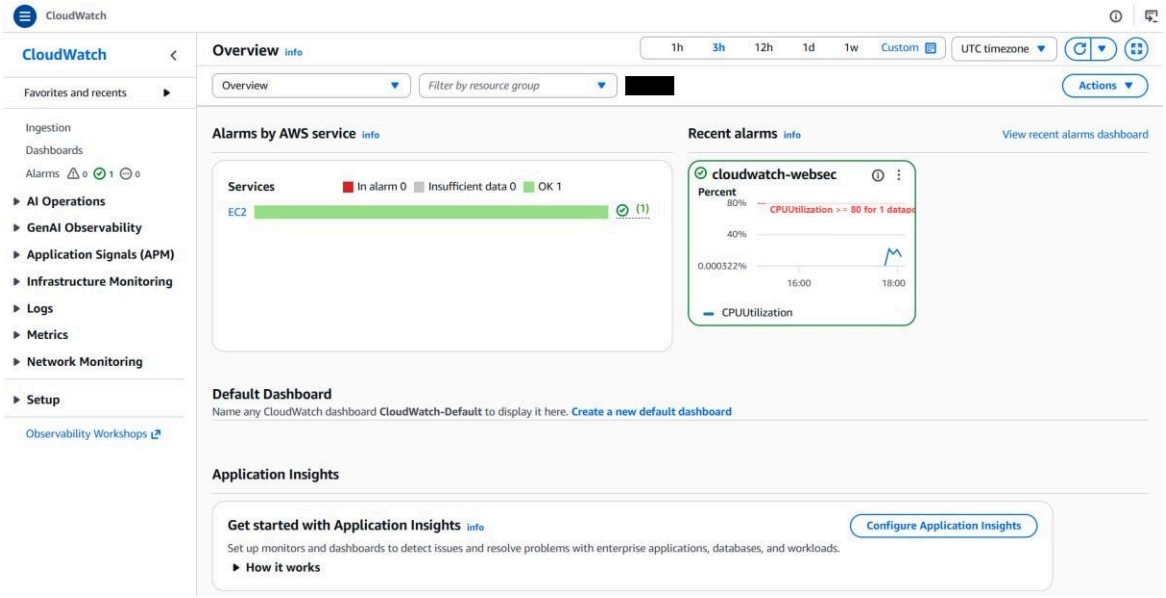
19. AWS KMS

AWS Key Management Service (KMS) provides encryption keys for multiple AWS services used in this project. The Amazon RDS instance is encrypted using the AWS-managed **aws/rds** KMS key, ensuring that database storage, automated backups, and snapshots are encrypted at rest. Additionally, AWS-managed KMS keys are available for services such as **AWS Secrets Manager (aws/secretsmanager)** and **Amazon SNS (aws/sns)**, providing encryption support for stored secrets and notification data where applicable. These AWS-managed keys eliminate the need for manual key administration while maintaining secure encryption across AWS services.



20. Amazon CloudWatch

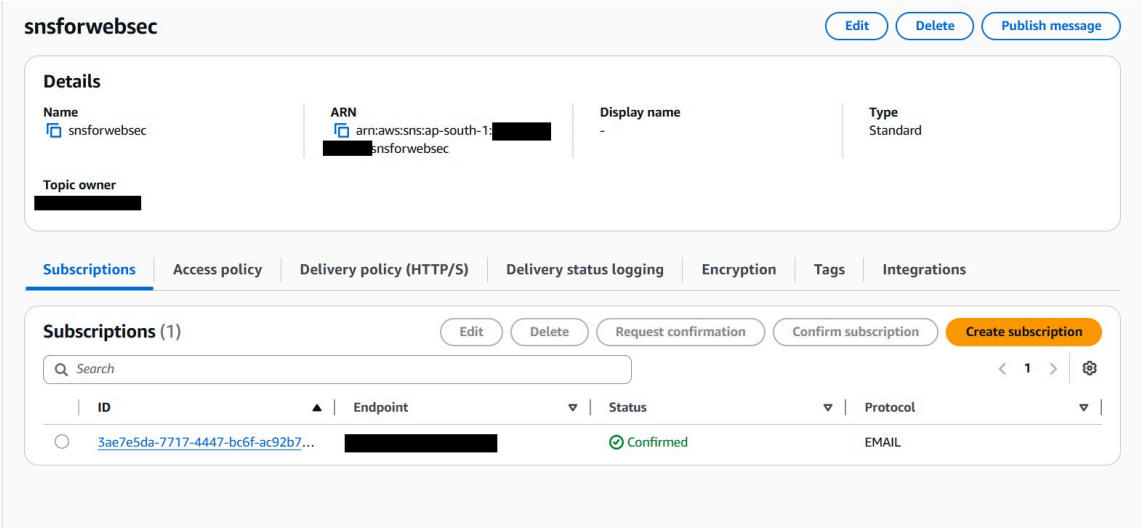
CloudWatch monitors EC2 CPU utilization for the web tier and raises an alarm when usage crosses an 80% threshold, giving early warning of load that may require scaling.



CloudWatch — alarms and dashboard overview

21. Amazon SNS

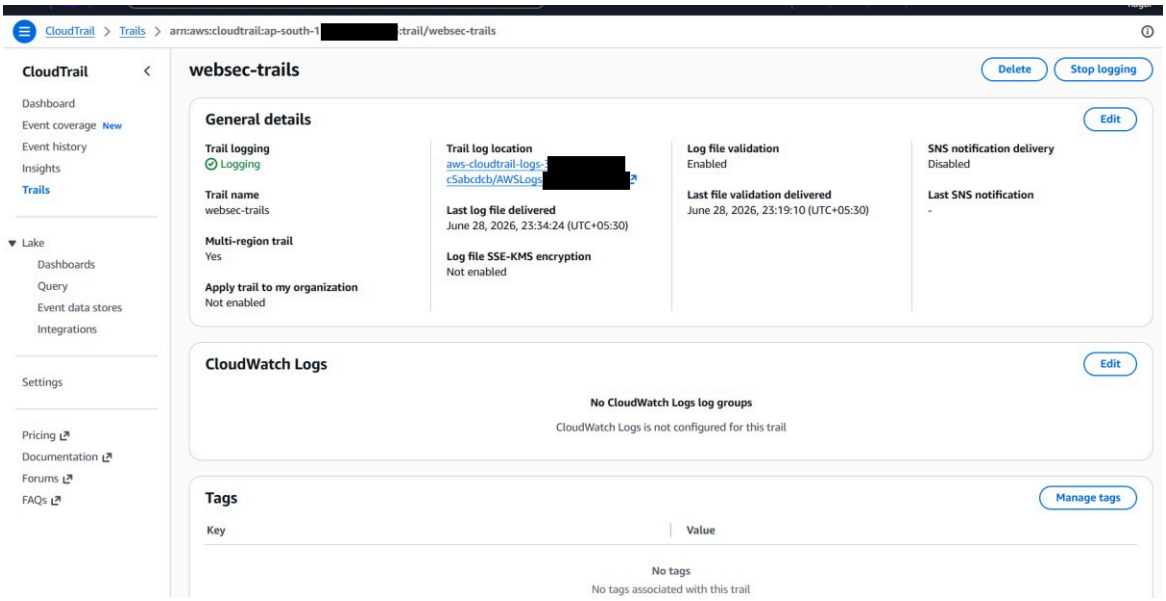
When a CloudWatch alarm fires, it publishes to an SNS topic that immediately emails the operations team, closing the loop between detection and human awareness.



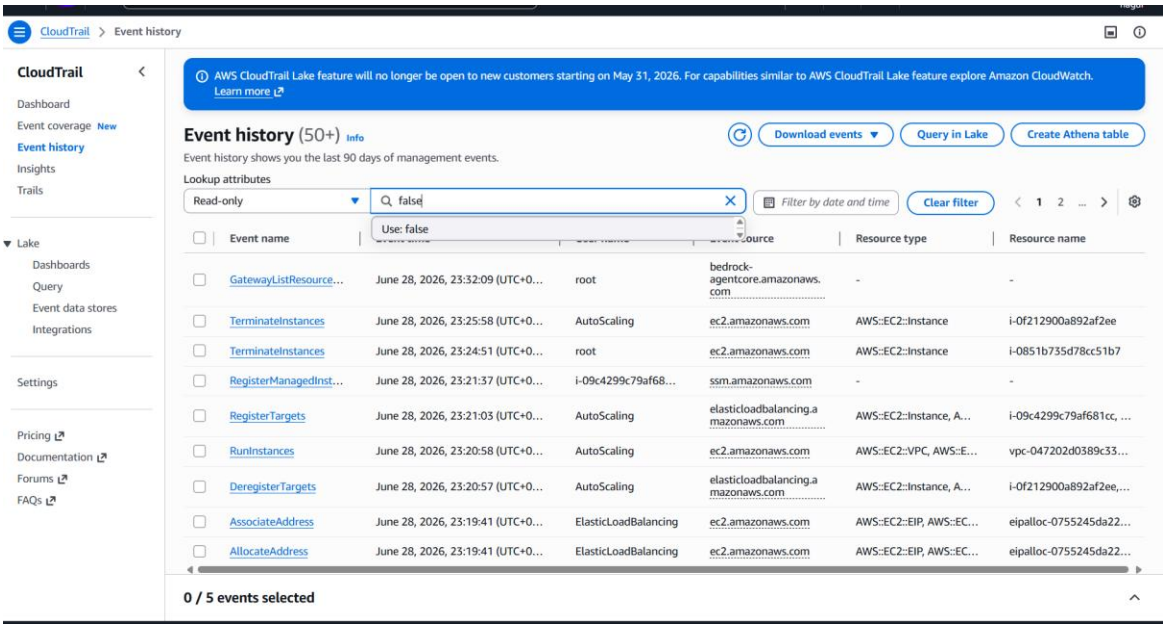
SNS topic — snsforwebsec subscription

22. AWS CloudTrail

CloudTrail records every API call made against the AWS account — who made it, when, and from where — providing a complete audit trail for security review and compliance.



CloudTrail — websec-trails configuration



CloudTrail — event history

23. Amazon S3

CloudTrail logs are delivered to a dedicated S3 bucket, providing durable, centralized storage for audit history that can be reviewed long after the events occurred.

24. Traffic Flow

Application Request Flow

1. Client sends an HTTP request over the internet.
2. Request enters the VPC through the Internet Gateway.
3. AWS WAF inspects and filters the request.
4. The Application Load Balancer receives the filtered request.
5. The request is forwarded to the Target Group.
6. An EC2 Web Server processes the request via IIS.
7. The web server queries Amazon RDS MySQL over port 3306 if needed.
8. The response travels back through the same path to the client.

Administrative Access Flow

9. Administrator connects via RDP to the Bastion Host using its public IP.
10. From the Bastion Host, the administrator RDPs into a private web server.

25. Security Best Practices Applied

- Network segmentation across public, private-web, and private-database subnets.
- Least-privilege IAM roles instead of long-lived access keys on instances.
- Database deployed in a private subnet with no public endpoint.
- Credentials stored in AWS Secrets Manager rather than hard-coded in application config.
- AWS WAF protecting the application from common web exploits.
- AWS KMS encryption for data at rest on RDS.
- Multi-AZ RDS deployment for automatic failover.
- CloudTrail auditing of all API activity.
- CloudWatch monitoring with SNS alerting.
- Auto Scaling for resilience and elasticity.
- Bastion Host as the sole path for administrative access.

26. Challenges Faced

- Understanding how route tables determine traffic flow between public and private subnets.
- Correctly scoping security group rules so each tier only accepts traffic from the tier above it.
- Attaching the right IAM permissions to the EC2 role without over-granting access.
- Configuring the Launch Template to consistently produce working instances from the custom AMI.

27. Future Improvements

- Add HTTPS using an AWS Certificate Manager (ACM) certificate and an HTTPS listener.
- Enable AWS Shield Advanced for enhanced DDoS protection.
- Replace the Bastion Host with AWS Systems Manager Session Manager for keyless administrative access.
- Enable AWS Config for continuous compliance monitoring.
- Enable Amazon GuardDuty for threat detection.
- Enable VPC Flow Logs for deeper network visibility.
- Migrate the manual setup to Infrastructure as Code using Terraform or AWS CloudFormation.
- Build a CI/CD pipeline with GitHub Actions for automated deployment.

28. Conclusion

This project delivered a secure, highly available, three-tier web application on AWS, built around network segmentation, least-privilege access, encryption at rest, and continuous monitoring. Working through VPC design, security group scoping, IAM permissions, Auto Scaling, and credential management provided hands-on experience with the core practices of cloud security engineering, and produced an architecture that reflects real-world production patterns rather than a simplified lab exercise.